
Ch09

01. 참조

02. 포인터

03. 배열과 포인터

04. 메모리

메모리 관리

- C++ 프로그램은 실행할 때부터 메모리의 특정 위치를 사용
- C++ 환경은 메모리를 효율적으로 관리하기 위해서 다음과 같이 메모리를 여러 영역으로 나눔

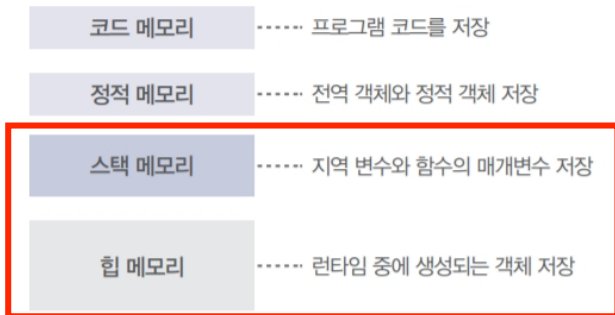


그림 9-22 프로그램이 사용하는 메모리 영역

스택 메모리(1)

- 스택 메모리(stack memory)는 함수의 지역 변수와 매개변수를 보유 parameter == local variable
- 스택이란 후입선출(last-in-first-out) 컨테이너로 마지막에 넣은 것이 가장 먼저 꺼내지는 자료 구조
- 스택의 이러한 특성은 함수의 지역 변수와 매개변수를 저장하는 데 적합
- [그림 9-23]은 main 함수에서 first 함수를, first 함수에서 second 함수를 호출했을 때 스택 메모리의 동작을 나타냄

스택 메모리(2)

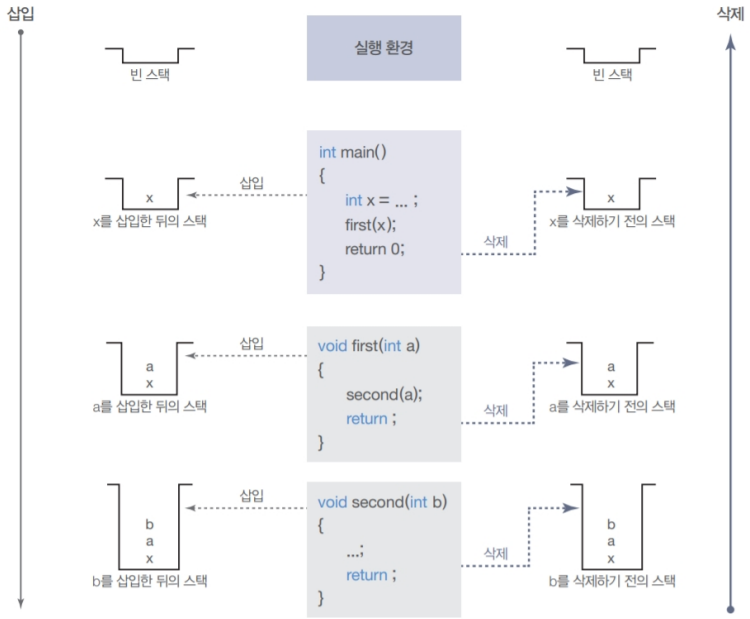


그림 9-23 스택 메모리의 동작

스택 메모리(3)

장점

- 시스템은 모든 프로그램에 이와 같은 스택 메모리를 사용
- 스택 메모리의 후입선출은 굉장히 효율적으로 작동
- 객체는 함수 스코프 내부에 있을 때만 스택 메모리에 유지되며, 함수의 스코프를 벗어나면 삭제되어서 더 이상 접근할 수 없게 됨

스택 메모리(4)

제한

- 스택 메모리를 사용하면 효율적이지만, 다음과 같은 2가지 제한이 발생
- 컴파일 시점에 객체의 이름을 정의해야 함
- 이름이 없는 객체는 스택에 저장할 수 없음
- 컴파일 시점에 객체의 크기를 정의해야 함
- 시스템이 객체의 크기를 모르면 스택 메모리에 영역을 할당할 수 없음
- 단일 객체의 경우는 자료형을 기반으로 크기를 확인하고 객체 리스트(배열 등)의 경우는 컴파일 때에 반드시 요소의 수를 정의해야 함

스택 메모리(5)

예시

- 메모리에 저장할 모든 객체는 컴파일 시점에 확실하게 이름과 크기가 정의되어 있어야 함
- 만약 다음과 같이 프로그램을 작성하면 오류가 발생

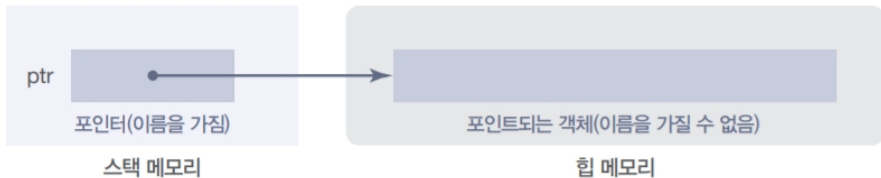
```
#include <iostream>
using namespace std;

int main ()
{
    int size;
    cin >> size;
    double array [size];           // Compilation error
    ...
    return 0;
}
```

힙 메모리(1)

- 프로그램을 컴파일하는 시점에 객체의 크기를 모른다면, 런타임 시점에 메모리에 객체를 만들어야 함
- 힙 메모리(heap memory)는 런타임 시점에 객체를 저장할 때 사용
- 또한 힙 메모리는 여유 메모리(free memory) 또는 동적 메모리(dynamic memory)라고도 부름

힙 메모리(2)



❗ 포인터는 스택 메모리에, 포인트되는 객체는 힙 메모리에 있음

그림 9-24 스택에 있는 포인터와 힙에 있는 포인트되는 객체

```
int* ptr;  
ptr = new int[size];  
ptr[0] = -50;  
ptr[3] = 7;
```

힙 메모리(3)

new와 delete 연산자

- 그럼 런타임 시점에 어떤 코드로 힙에 객체를 만들 수 있을까?
- 그리고 사용을 모두 완료해서 더 이상 필요가 없어진 힙에 있는 객체를 어떻게 제거할 수 있을까?

표 9-6 힙에 메모리 할당과 해제를 위한 연산자

그룹	이름	연산자	표현식	우선 순위	결합방향
단항	객체 할당	new	new type	17	←
	배열 할당	new[]	new type[size]		
	객체 제거	delete	delete ptr		
	배열 제거	delete[]	delete[] ptr		

```
int* p = new int;  
*p = 50;  
delete p; p=NULL;
```

```
int size = 10; // or from user input  
int* A = new int[size];  
*(A+5) = 77; A[3] = 99;  
delete[] A; A=NULL;
```

힙 메모리(4)

- 첫 번째 연산자는 하나의 객체를 힙에 생성할 때 사용
- 두 번째 연산자는 힙에 객체 배열을 생성할 때 사용
- 세 번째 연산자는 포인터를 사용해서 힙에 있는 객체를 하나 제거할 때 사용
- 네 번째 연산자는 포인터를 사용해서 힙에 있는 객체 배열을 제거할 때 사용

힙 메모리(5)



그림 9-25 new 연산자와 delete 연산자

힙 메모리(6)

예시

- 이번에는 힙에 객체를 만들고 사용자의 입력을 크기로 하는 배열을 알아봄
- 컴파일할 때 배열의 크기가 정의되지 않으므로 이러한 배열은 스택 메모리에 생성할 수 없음
- 따라서 [그림 9-26]과 같은 형태로 런타임 중에 생성해야 함

힙 메모리(7)

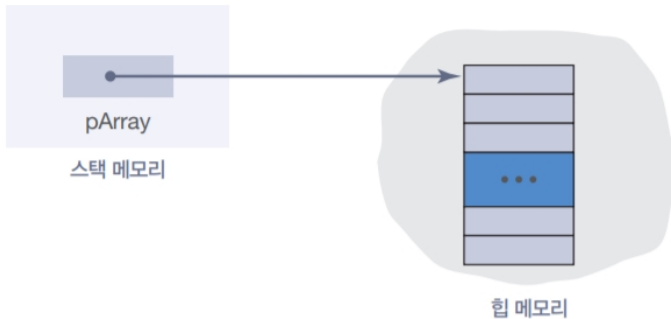


그림 9-26 스택에 포인터 만들고, 힙에 배열을 만드는 구성

힙 메모리(8)

프로그램 9-14 힙 메모리에 크기가 유동적인 배열 만들기

```
1  /*****
2  * The program shows how to create and access a variable-size *
3  * array in the heap and use pointers to access elements.      *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Declaration of array size and the pointer in the stack
11     int size;
12     int* pArray;
13     // Validation of the size to be greater than zero
14     do
15     {
16         cout << "Enter the array size (larger than zero): ";
17         cin >> size;
18     } while (size <= 0);
19     // Creation of array in the heap
20     pArray = new int [size];
```

힙 메모리(9)

프로그램 9-14 힙 메모리에 크기가 유동적인 배열 만들기

```
21 // Inputting the values of array
22 for (int i = 0; i < size ; i++)
23 {
24     cout << "Enter the value for element " << i << ": ";
25     cin >> *(pArray + i);
26 }
27 // Outputting the values of the array
28 cout << "The elements in the array are: " << endl;
29 for (int i = 0; i < size ; i++)
30 {
31     cout << *(pArray + i) << " ";
32 }
33 // Deleting the array in the heap
34 delete [ ] pArray;
35 return 0;
36 }
```

힙 메모리(10)

프로그램 9-14 힙 메모리에 크기가 유동적인 배열 만들기

Run:

Enter the array size (larger than zero): 3

Enter the value for element 0: 6

Enter the value for element 1: 12

Enter the value for element 2: 5

The elements in the array are:

6 12 5

힙 메모리(11)

힙 메모리와 관련된 문제

- 힙 메모리 그리고 힙 메모리와 관련된 연산자(new와 delete)를 사용할 때는 주의할 점이 있음

메모리 할당 없이 제거

- 가장 대표적으로 발생할 수 있는 문제는 new 키워드를 사용하지 않았는데 delete 키워드를 사용하는 경우
- 즉 힙 메모리에 메모리 할당을 하지 않았는데 제거하려는 경우

```
double x = 23.4;  
double* pX = &x;           // Allocation in the stack  
delete pX;                  // Deletion in the heap may create run-time  
error
```

힙 메모리(12)

제거 없이 메모리 할당하기(메모리 누수)

- 메모리와 관련된 심각한 문제로 메모리 누수(memory leak)가 있음
- 메모리 누수는 new 키워드로 힙 메모리 위에 영역을 잡은 뒤, delete 키워드로 제거하지 않았을 때 발생

```
double* pX = new double;  
... // using the allocated memory
```

힙 메모리(13)

허상 포인터

- 또 다른 문제로 허상 포인터(dangling pointer)가 있음
- 허상 포인터는 포인터가 가리키고 있는 객체를 delete 키워드로 제거했는데, 포인터를 활용하려고 할 때 발생

```
double* pX = new double;  
...           // using the allocated memory  
delete pX;  
*pX = 35.3; // Dangling pointer
```